

Fig. 13: FFTW (Fastest Fourier Transform in the West) on heterogeneous compute nodes

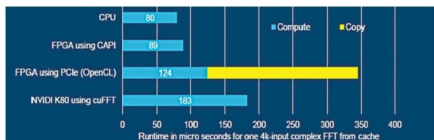


Fig. 14: Latency

the application executes on the host processor, the CAPI model offloads computation-heavy functions to the accelerator.

The accelerator is a full peer to the application. It uses an unmodified virtual address with full access to the application's address space. It uses the processor's page tables directly with page faults handled by system software. For a CAPI solution, the application might set up the data for the AFU. The application sends the AFU a process element to initiate the data-intensive execution.

The process element contains a work element descriptor (WED) provided by the application. The WED can contain the full description of the job to be performed or a pointer to other main memory structures in the application's memory space. The application can be the master or the slave to the accelerator—whichever mechanism is demanded by the algorithm. Alternatively, the accelerator can receive incoming packets from the network, work on the data, and then

inform the application that the processed data is ready for consumption.

The acceleration platform can be integrated into cloud-based services. In this case, the application on the core runs as an Application as a Service (AaaS) for other applications that require services of the accelerator. The AaaS model maintains work queues for the accelerator on behalf of each of the requesting threads and performs any maintenance function needed to inform the accelerator of pending work. These connections are made through operating-system kernel extensions and library functions created specifically for CAPI.

Operating-system kernel extensions will be made available to all Power-Linux Little Endian distributions starting with Canonical's Ubuntu. When the system boots with a CAPI device connected on a PCIe port, kernel extensions recognise the CAPI device. These perform personalised system initialisations, including configuration of the CAPP unit connection to the PCIe port.

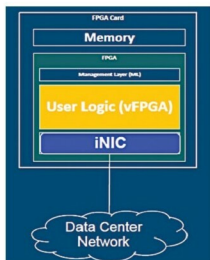


Fig. 15: Standalone FPGA accelerator

Additional kernel extensions perform maintenance, system protection and communication functions. The client application uses a CAPI library called 'libxccl', which provides multiple functions for the application to connect, call, communicate and disconnect with an available CAPI device. libxccl calls include many basic functions, such as those that allow the application to provide the virtual storage address of data structures to the accelerator. It also provides a memory-mapped I/O interface for the application to read/write registers in the accelerator's register space on the FPGA.

Prior to CAPI, an application calls the device driver to utilise the FPGA accelerator. The device driver performs a memory mapping operation. CAPI connects a custom acceleration engine to the coherent fabric of the Power9 chip. Because FPGAs are reconfigurable, hardware can be customised without the traditional costs of hardware fabrication.

A standalone appliance/accelerator is equipped with an FPGA, (optional) local memory and an integrated network controller interface (iNIC). The iNIC enables the FPGA to hook itself to the network and to communicate with other DC resources, such as servers, disks, I/O and other FPGA appliances.

*(To be continued next month)*